Parallel Collatz Verification via the Optional Intervals Event Framework

Kedar Yadav

May 2026

1 Overview

Li et al. [1] introduce an algebraic framework for event execution based on Optional Intervals Events (OIEs). In this framework, independent (parallel) events are combined using *complete sequential addition*, denoted by

$$\oplus,$$

while ordered stages are composed using *complete sequential multiplication*, denoted by

$$\otimes.$$

This formalism provides a natural way to describe the structure of the MPI-based Collatz verification algorithm, where independent worker processes execute concurrently, followed by a global reduction step.

2 Collatz Verification Problem

For a positive integer n, define the Collatz map

$$T(n) = \begin{cases} \dfrac{n}{2}, & n \equiv 0 \pmod 2, \\ 3n+1, & n \equiv 1 \pmod 2. \end{cases}$$

Given an interval $[L, U]$, the computational task is to verify that each seed $n \in [L, U]$ reaches 1, subject to a finite iteration cutoff.

Define the event

$$E_n := \text{“verify the trajectory starting at seed } n\text{”}.$$

3 Serial Structure

In the standard serial implementation, all seeds are processed sequentially:

$$\mathcal{C}_{\text{serial}} = E_L \otimes E_{L+1} \otimes \cdots \otimes E_U.$$

This introduces a strict ordering that is not required by the mathematics of the problem, since each trajectory is independent (aside from optional pruning heuristics).

4 MPI Decomposition

The interval $[L, U]$ is partitioned into P disjoint subintervals:

$$[L, U] = \bigcup_{r=0}^{P-1} I_r, \qquad I_i \cap I_j = \varnothing \quad (i \neq j).$$

Each MPI rank r processes its assigned block I_r. The corresponding worker event is

$$W_r = \bigotimes_{n \in I_r} E_n.$$

While each worker executes serially, the workers themselves are independent. Therefore,

$$W_0 \oplus W_1 \oplus \cdots \oplus W_{P-1}.$$

The full computation is thus

$$\mathcal{C}_{\text{MPI}} = \text{Partition} \otimes \left(\bigoplus_{r=0}^{P-1} W_r \right) \otimes \text{Reduction}.$$

This directly mirrors the MPI implementation:

- Partition: distribute seeds across ranks,
- Parallel phase: each rank verifies its subset,
- Reduction: combine results via MPI.

5 Performance Model

Let T_{serial} denote the runtime of the serial implementation and $T_{\text{MPI}}(P)$ the runtime using P processes.

The MPI runtime can be approximated as

$$T_{\text{MPI}}(P) \approx T_{\text{partition}} + \max_{0 \leq r < P} T(W_r) + T_{\text{reduction}}.$$

Under ideal load balancing and negligible communication overhead,

$$T_{\text{MPI}}(P) \approx \frac{T_{\text{serial}}}{P}.$$

Define speedup and efficiency as

$$S(P) = \frac{T_{\text{serial}}}{T_{\text{MPI}}(P)}, \qquad \eta(P) = \frac{S(P)}{P}.$$

In practice, performance is limited by:

- load imbalance between intervals,
- process startup overhead,
- memory hierarchy effects,
- synchronization and reduction costs.

6 Interpretation

The OIE framework clarifies the structural distinction:

$$\text{Serial Collatz} \;=\; \text{pure} \;\otimes\; \text{chain},$$

$$\text{MPI Collatz} \;=\; \text{setup} \;\otimes\; \text{parallel layer } (\oplus) \;\otimes\; \text{reduction}.$$

Thus, the MPI algorithm realizes a hybrid event structure in which concurrency is explicitly isolated in the $\oplus$ layer, while unavoidable ordering is confined to $\otimes$ stages.

7 Conclusion

The OIE formalism does not modify the Collatz dynamics, but provides a precise algebraic description of the computational structure. In particular, it identifies the exact level at which parallelism can be introduced without altering correctness, which is realized in the MPI implementation.

References

[1] Z. Li, Y. Gong, L. Yu, Y. Cao, and B. Yin, *Optional intervals event, sequential operation and their applications in physics, computer science and applied mathematics*, arXiv:2504.09471, 2025. `https://arxiv.org/abs/2504.09471`